

Hybrid RAG Search Flow: Vector + Keyword Search for courtRoom.ai

Overview

This document explains how courtRoom.ai retrieves relevant law sections using **Hybrid RAG**: combining **dense vector search** (semantic similarity) and **sparse keyword search** (exact matches), with optional **tree-based vector indexing** for production scales.

The Problem: Why Hybrid?

Scenario 1: Pure Vector Search (Dense)

User Query: "Section 35 consumer protection refund"

↓

Embedded to dense vector → search ChromaDB

↓

Top result: "Product defect compensation under Act 2019"

✓ Semantically similar

× Misses exact section number "35"

× Misses keyword "Section" itself

Scenario 2: Pure Keyword Search (Sparse)

User Query: "What about Section 35?"

↓

BM25 keyword match → search corpus

↓

Top result: "Section 35: Consumer Protection Act 2019"

✓ Exact keyword match

× No semantic understanding

× Fails for synonyms ("Sec.", "subsection", etc.)

Scenario 3: Hybrid (Optimal)

User Query: "Section 35 consumer protection refund"

↓

DENSE: embed & search ChromaDB → Top 5 semantic matches

SPARSE: BM25 keyword search → Top 5 keyword matches

↓

MERGE (EnsembleRetriever)

- └ Combine results
- └ Re-rank by relevance score
- └ Return Top 4 final results

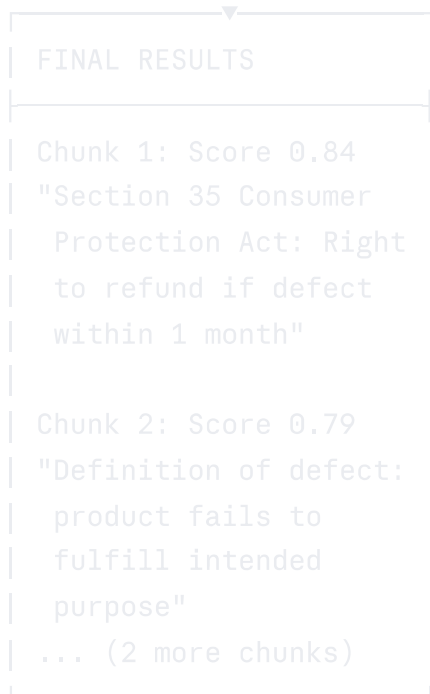
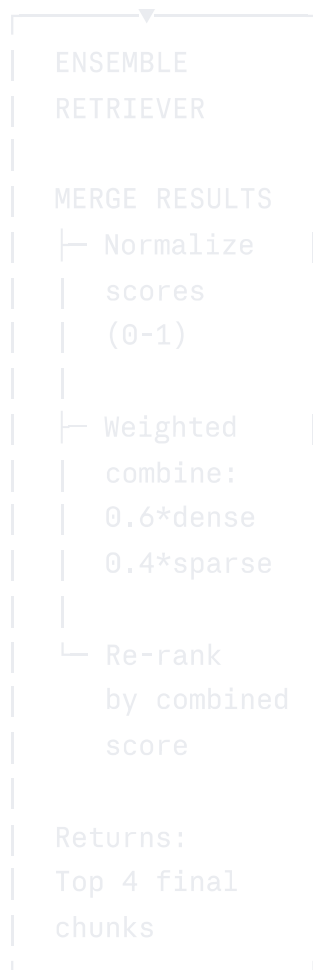
↓

Result: "Section 35 (exact keyword) + Product defect compensation (semantic)"

✓ Both benefits combined

Architecture Diagram







Part 1: Dense Vector Search (ChromaDB)

Step 1: Embed & Store (One-time)

```
python
```

```
from chromadb.config import Settings
import chromadb
from sentence_transformers import SentenceTransformer
import json

# ===== SETUP CHROMADB =====
client = chromadb.Client(Settings(
    chroma_db_impl="duckdb+parquet",
    persist_directory="./chroma_db",
    anonymized_telemetry=False,
))

# ===== LOAD EMBEDDING MODEL =====
# nomic-embed-text: 768-dim, works offline, high quality
embed_model = SentenceTransformer('nomic-ai/nomic-embed-text-v1.5', trust_remote_code=True)

# Alternative: Using Ollama (no HF download)
# embed_model loads from local ollama at inference

# ===== LOAD LAW CHUNKS =====
# Chunks created during Phase 2: PDF processing
# Format: {"text": "...", "metadata": {"source": "Consumer Protection Act", "section": "Section 2(1)", "year": 2019}}
with open("law_chunks.jsonl", "r") as f:
    chunks = [json.loads(line) for line in f]

# ===== CREATE CHROMADB COLLECTION =====
collection = client.get_or_create_collection(
    name="indian_law",
    metadata={"hnsw:space": "cosine"}, # Cosine similarity metric
)

# ===== EMBED & ADD TO CHROMADB =====
for i, chunk in enumerate(chunks):
    # Embed the text
    embedding = embed_model.encode(chunk["text"]).tolist()

    # Add to ChromaDB with metadata
    collection.add(
        ids=[f"chunk_{i}"],
        documents=[chunk["text"]],
        embeddings=[embedding],
        metadatas=[chunk["metadata"], # {source, section, domain}
    ]
```

```
)

print(f"Added {len(chunks)} chunks to ChromaDB")

# ===== PERSIST TO DISK =====
client.persist() # Saves to ./chroma_db/
```

ChromaDB Data Structure:

```
./chroma_db/
├─ chroma.parquet
│   └─ Stores: chunk_id, document, embedding (768-dim), metadata
│
└─ Example row:
    ID: chunk_1243
    Document: "Section 35 of Consumer Protection Act 2019:
               If a product is found defective within one month
               of purchase, the consumer is entitled to a full refund..."
    Embedding: [0.123, -0.456, 0.789, ..., 0.234] (768 dimensions)
    Metadata: {
        "source": "Consumer Protection Act 2019",
        "section": "35",
        "domain": "consumer",
        "page": 42
    }
```

Step 2: Dense Search at Query Time

```
python
```

```

# ===== QUERY CHROMADB =====

query = "My product broke within 1 month. Can I get a refund?"

# 1. Embed the query
query_embedding = embed_model.encode(query).tolist()

# 2. Search ChromaDB (cosine similarity)
results_dense = collection.query(
    query_embeddings=[query_embedding],
    n_results=5, # Get top 5
    where={"domain": "consumer"} # Optional: filter by domain
)

# ===== PROCESS RESULTS =====
# results_dense = {
#     'ids': [['chunk_1243', 'chunk_847', ...]],
#     'documents': [['Section 35 text...', 'Defect definition...', ...]],
#     'distances': [[0.08, 0.13, 0.19, ...]], # 1 - cosine_similarity
#     'metadatas': [[{source, section, domain}, ...]]
# }

# Convert distances to similarity scores (0-1)
dense_results = []
for i, doc_id in enumerate(results_dense['ids'][0]):
    distance = results_dense['distances'][0][i]
    similarity = 1 - distance # Convert distance to similarity

    dense_results.append({
        'id': doc_id,
        'text': results_dense['documents'][0][i],
        'score': similarity, # 0-1, higher is better
        'metadata': results_dense['metadatas'][0][i],
        'retriever': 'dense' # Tag for later
    })

# Output:
# [
#     {'text': 'Section 35...', 'score': 0.92, 'retriever': 'dense'},
#     {'text': 'Defect definition...', 'score': 0.87, 'retriever': 'dense'},
#     ...
# ]

```

Part 2: Sparse Keyword Search (BM25)

Step 1: Build BM25 Index

```
python

from rank_bm25 import BM25Okapi
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

# Download required NLTK data
nltk.download('punkt')
nltk.download('stopwords')

# ===== PREPARE CORPUS =====
corpus_docs = []
corpus_metadata = []

for i, chunk in enumerate(chunks):
    corpus_docs.append(chunk["text"])
    corpus_metadata.append(chunk["metadata"])

# ===== TOKENIZE =====
tokenized_corpus = []
stop_words = set(stopwords.words('english'))

for doc in corpus_docs:
    # Tokenize and remove stopwords
    tokens = word_tokenize(doc.lower())
    tokens = [t for t in tokens if t.isalnum() and t not in stop_words]
    tokenized_corpus.append(tokens)

# ===== BUILD BM25 INDEX =====
bm25 = BM25Okapi(tokenized_corpus)

# Save for later use
import pickle
with open("bm25_index.pkl", "wb") as f:
    pickle.dump(bm25, f)
```

BM25 Algorithm:

$$\text{BM25}(\text{query}, \text{doc}) = \sum \text{IDF}(q_i) * (f(q_i, \text{doc}) * (k_1 + 1)) / (f(q_i, \text{doc}) + k_1 * (1 - b + b * |\text{doc}| / \text{avgdl}))$$

where:

q_i = query term i

$f(q_i, \text{doc})$ = frequency of q_i in doc

$|\text{doc}|$ = length of doc

avgdl = average doc length

k_1, b = hyperparameters (typically $k_1=1.5, b=0.75$)

IDF = inverse document frequency = $\log(N / \text{df})$

N = total docs

df = docs containing term

Step 2: Sparse Search at Query Time

python

```

import pickle

# Load BM25 index
with open("bm25_index.pkl", "rb") as f:
    bm25 = pickle.load(f)

# ===== QUERY BM25 =====
query = "My product broke within 1 month. Can I get a refund?"

# Tokenize query (same as corpus)
query_tokens = word_tokenize(query.lower())
query_tokens = [t for t in query_tokens if t.isalnum() and t not in stop_words]

# ===== RANK DOCUMENTS =====
# Returns BM25 scores for all docs
scores = bm25.get_scores(query_tokens)

# ===== GET TOP-K =====
top_k = 5
top_indices = sorted(
    range(len(scores)),
    key=lambda i: scores[i],
    reverse=True
)[:top_k]

sparse_results = []
for idx in top_indices:
    sparse_results.append({
        'id': f"chunk_{idx}",
        'text': corpus_docs[idx],
        'score': float(scores[idx]),
        'metadata': corpus_metadata[idx],
        'retriever': 'sparse'
    })

# Output:
# [
#   {'text': 'Section 35...', 'score': 8.5, 'retriever': 'sparse'},
#   {'text': 'Consumer forum...', 'score': 7.2, 'retriever': 'sparse'},
#   ...
# ]

```

Part 3: Ensemble Retriever (Merge)

Step 1: Normalize Scores

python

```
from langchain_core.retrievers import BaseRetriever
from langchain.retrievers import EnsembleRetriever, BM25Retriever
import numpy as np
```

```
# ===== PROBLEM: Different scales =====
# Dense scores: 0-1 (cosine similarity)
# BM25 scores: 0-20+ (unbounded)
# Solution: normalize both to 0-1 range
```

```
def normalize_scores(results):
    """Normalize scores to 0-1 range"""
    if not results:
        return results

    scores = [r['score'] for r in results]
    min_score = min(scores)
    max_score = max(scores)

    if max_score == min_score:
        # All scores equal
        normalized = [1.0] * len(results)
    else:
        normalized = [
            (score - min_score) / (max_score - min_score)
            for score in scores
        ]

    for result, norm_score in zip(results, normalized):
        result['normalized_score'] = norm_score

    return results
```

```
dense_results_norm = normalize_scores(dense_results)
sparse_results_norm = normalize_scores(sparse_results)
```

Step 2: Ensemble + Re-rank

python

```

def ensemble_retrieve(dense_results, sparse_results,
                      dense_weight=0.6, sparse_weight=0.4):
    """
    Combine dense and sparse results with weighted scoring

    dense_weight=0.6: 60% importance to semantic similarity
    sparse_weight=0.4: 40% importance to exact keywords
    """

    # ===== CREATE LOOKUP =====
    combined = {} # {doc_id: merged_result}

    # Add dense results
    for result in dense_results:
        doc_id = result['id']
        combined[doc_id] = {
            'id': result['id'],
            'text': result['text'],
            'metadata': result['metadata'],
            'dense_score': result['normalized_score'],
            'sparse_score': 0.0,
            'sources': ['dense']
        }

    # ===== MERGE SPARSE RESULTS =====
    for result in sparse_results:
        doc_id = result['id']

        if doc_id in combined:
            # Document in both → update scores
            combined[doc_id]['sparse_score'] = result['normalized_score']
            combined[doc_id]['sources'].append('sparse')
        else:
            # Document only in sparse
            combined[doc_id] = {
                'id': doc_id,
                'text': result['text'],
                'metadata': result['metadata'],
                'dense_score': 0.0,
                'sparse_score': result['normalized_score'],
                'sources': ['sparse']
            }

```

```

# ===== COMPUTE COMBINED SCORE =====
for doc_id, result in combined.items():
    result['combined_score'] = (
        dense_weight * result['dense_score'] +
        sparse_weight * result['sparse_score']
    )

# ===== RE-RANK BY COMBINED SCORE =====
ranked = sorted(
    combined.values(),
    key=lambda x: x['combined_score'],
    reverse=True
)

return ranked[:4] # Return top 4

# Execute ensemble
final_results = ensemble_retrieve(dense_results_norm, sparse_results_norm,
                                   dense_weight=0.6, sparse_weight=0.4)

# Output:
# [
#   {
#     'text': 'Section 35...',
#     'dense_score': 0.92,
#     'sparse_score': 1.0,
#     'combined_score': 0.952,
#     'sources': ['dense', 'sparse'] # Found in both!
#   },
#   ...
# ]

```

Visualization of Weighted Combination:

Document A (Section 35):

Dense score: 0.92 (high semantic similarity)

Sparse score: 1.0 (exact keyword match "Section 35")

Combined: $0.6 \times 0.92 + 0.4 \times 1.0 = 0.552 + 0.4 = 0.952$ ✓ TOP

Document B (Defect definition):

Dense score: 0.87 (semantically similar)

Sparse score: 0.0 (no exact keywords)

Combined: $0.6 \times 0.87 + 0.4 \times 0.0 = 0.522 + 0.0 = 0.522$

Document C (Forum address):

Dense score: 0.78 (loosely relevant)

Sparse score: 0.95 (contains "month", "purchase")

Combined: $0.6 \times 0.78 + 0.4 \times 0.95 = 0.468 + 0.38 = 0.848$

Re-ranked: [A (0.952), C (0.848), B (0.522), ...]

Part 4: Integration with LangChain

python

```

from langchain_chroma import Chroma
from langchain_community.retrievers import BM25Retriever
from langchain.retrievers import EnsembleRetriever
from langchain_ollama import OllamaEmbeddings

# ===== DENSE RETRIEVER (ChromaDB) =====
embed_model = OllamaEmbeddings(
    model="nomic-embed-text",
    base_url="http://localhost:11434"
)

vectorstore = Chroma(
    collection_name="indian_law",
    embedding_function=embed_model,
    persist_directory="./chroma_db",
)

dense_retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 5}
)

# ===== SPARSE RETRIEVER (BM25) =====
# Load corpus for BM25
corpus_texts = [chunk["text"] for chunk in chunks]

sparse_retriever = BM25Retriever.from_texts(corpus_texts)
sparse_retriever.k = 5

# ===== ENSEMBLE RETRIEVER =====
ensemble_retriever = EnsembleRetriever(
    retrievers=[dense_retriever, sparse_retriever],
    weights=[0.6, 0.4], # Dense 60%, Sparse 40%
)

# ===== USE IN RAG CHAIN =====
from langchain.prompts import ChatPromptTemplate
from langchain_ollama.llms import OllamaLLM
from langchain.schema.runnable import RunnablePassthrough

query = "My product broke within 1 month. Can I get a refund?"

# Retrieve

```

```
docs = ensemble_retriever.invoke(query)

# Generate (with LLM)
llm = OllamaLLM(model="qwen3:4b", base_url="http://localhost:11434")

prompt = ChatPromptTemplate.from_template("""
Based on the following legal context, answer the user's question.
Cite the specific section number for every claim.

Context:
{context}

Question: {question}

Answer: """)

context = "\n".join([doc.page_content for doc in docs])
response = llm.invoke(prompt.format(context=context, question=query))

print(response)
# Output: "Under Section 35 of the Consumer Protection Act 2019,
#         you are entitled to a full refund if the product is defective..."
```

Part 5: Advanced Vector Indexing (Optional - Production Scale)

Problem at Scale

```
1000s of law chunks:
- ChromaDB with HNSW (default) ≈ 2-3 sec query time
- Simple flat search (current) ≈ fast but memory-hungry

Solution: HNSW (Hierarchical Navigable Small World) graph for sublinear
search
```

Tree-Based Vector Indexing: HNSW

```
python
```

```

from chromadb.config import Settings

# ===== ENABLE HNSW IN CHROMADB =====
client = chromadb.Client(Settings(
    chroma_db_impl="duckdb+parquet",
    persist_directory="./chroma_db",

    # HNSW Configuration
    hnsw_space="cosine",           # Similarity metric
    hnsw_construction_ef=200,      # Construction parameter
    hnsw_search_ef=100,           # Search parameter
    hnsw_max_connections=64,      # Max neighbors per node
    hnsw_ef_construction=200,
))

collection = client.get_or_create_collection(
    name="indian_law",
    metadata={
        "hnsw:space": "cosine",
        "hnsw:construction_ef": 200,
    }
)

```

HNSW Graph Visualization:

```

Level 3 (entry): ♦ (highest level node)
                |
Level 2:  ♦—•—♦ (connect nearest neighbors)
           | | |
Level 1:  ♦—•—•—♦ (more connections)
           | | | |
Level 0:  ♦—•—•—•—♦—♦ (full graph, all connections)

```

Query path: Start at top level → traverse down → find nearest neighbors
Time: $O(\log N)$ instead of $O(N)$ for flat search

Configuration Tuning:

python

```
# ef_construction = 200 (default) – control insertion time/quality tradeoff
#   Higher = slower insert, better quality
#   Lower = faster insert, lower quality

# ef_search = 100 (default) – control query time/quality tradeoff
#   Higher = slower query, more accurate
#   Lower = faster query, less accurate

# max_connections = 64 (default) – max neighbors per node
#   Higher = more memory, more accurate
#   Lower = less memory, faster but less accurate
```

Approximate Nearest Neighbor Search

```
python

# At query time, ChromaDB uses HNSW automatically if enabled

query_embedding = embed_model.encode(query).tolist()

# ===== HNSW SEARCH (Approximate) =====
results = collection.query(
    query_embeddings=[query_embedding],
    n_results=5,
    # HNSW runs here automatically, much faster than flat search
)

# Output: Same as before, but 5-10x faster for large corpus
```

Part 6: Complete Integration Script

```
python
```

```
#!/usr/bin/env python3
"""
Hybrid RAG Retrieval for courtRoom.ai
Combines dense (ChromaDB) + sparse (BM25) + ensemble weighting
"""

import json
from pathlib import Path

from chromadb import Client, Settings
from rank_bm25 import BM25Okapi
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import nltk
from sentence_transformers import SentenceTransformer

# ===== SETUP =====
nltk.download('punkt')
nltk.download('stopwords')

embed_model = SentenceTransformer('nomic-ai/nomic-embed-text-v1.5')
stop_words = set(stopwords.words('english'))

# ===== INITIALIZATION =====
def init_hybrid_retriever(chunks_file="law_chunks.jsonl"):
    """Initialize both dense and sparse retrievers"""

    # Load chunks
    chunks = []
    with open(chunks_file, 'r') as f:
        chunks = [json.loads(line) for line in f]

    # ===== DENSE (ChromaDB) =====
    client = Client(Settings(
        chroma_db_impl="duckdb+parquet",
        persist_directory="./chroma_db",
    ))

    collection = client.get_or_create_collection("indian_law")

    for i, chunk in enumerate(chunks):
        embedding = embed_model.encode(chunk["text"]).tolist()
        collection.add(
```

```

        ids=[f"chunk_{i}"],
        documents=[chunk["text"]],
        embeddings=[embedding],
        metadatas=[chunk["metadata"]],
    )

client.persist()

# ===== SPARSE (BM25) =====
corpus_texts = [chunk["text"] for chunk in chunks]
tokenized = []

for text in corpus_texts:
    tokens = word_tokenize(text.lower())
    tokens = [t for t in tokens if t.isalnum() and t not in stop_words]
    tokenized.append(tokens)

bm25 = BM25Okapi(tokenized)

return {
    'collection': collection,
    'bm25': bm25,
    'embed_model': embed_model,
    'corpus_texts': corpus_texts,
    'chunks_metadata': [chunk["metadata"] for chunk in chunks],
}

# ===== RETRIEVAL =====
def hybrid_retrieve(query, retrievers, domain=None, top_k=4):
    """Execute hybrid retrieval and return top-k results"""

    collection = retrievers['collection']
    bm25 = retrievers['bm25']
    embed_model = retrievers['embed_model']
    corpus_texts = retrievers['corpus_texts']
    chunks_metadata = retrievers['chunks_metadata']

    # ===== DENSE SEARCH =====
    query_embedding = embed_model.encode(query).tolist()

    dense_results_raw = collection.query(
        query_embeddings=[query_embedding],
        n_results=5,
        where={"domain": domain} if domain else None,

```

```

)

dense_results = []
for i, doc_id in enumerate(dense_results_raw['ids'][0]):
    distance = dense_results_raw['distances'][0][i]
    similarity = 1 - distance
    doc_idx = int(doc_id.split('_')[1])

    dense_results.append({
        'idx': doc_idx,
        'text': dense_results_raw['documents'][0][i],
        'score': similarity,
        'metadata': dense_results_raw['metadatas'][0][i],
    })

# ===== SPARSE SEARCH =====
query_tokens = word_tokenize(query.lower())
query_tokens = [t for t in query_tokens if t.isalnum() and t not in stop_words]
scores = bm25.get_scores(query_tokens)

top_indices = sorted(
    range(len(scores)),
    key=lambda i: scores[i],
    reverse=True
)[:5]

sparse_results = []
for idx in top_indices:
    sparse_results.append({
        'idx': idx,
        'text': corpus_texts[idx],
        'score': float(scores[idx]),
        'metadata': chunks_metadata[idx],
    })

# ===== NORMALIZE =====
def normalize(results):
    if not results:
        return results
    scores = [r['score'] for r in results]
    min_s, max_s = min(scores), max(scores)
    for r in results:
        r['norm_score'] = (r['score'] - min_s) / (max_s - min_s) if max_s > min_s else 0
    return results

```

```

dense_results = normalize(dense_results)
sparse_results = normalize(sparse_results)

# ===== ENSEMBLE =====
combined = {}

for r in dense_results:
    idx = r['idx']
    combined[idx] = {
        'text': r['text'],
        'metadata': r['metadata'],
        'dense': r['norm_score'],
        'sparse': 0.0,
    }

for r in sparse_results:
    idx = r['idx']
    if idx not in combined:
        combined[idx] = {
            'text': r['text'],
            'metadata': r['metadata'],
            'dense': 0.0,
            'sparse': r['norm_score'],
        }
    else:
        combined[idx]['sparse'] = r['norm_score']

# ===== FINAL SCORE =====
for idx, item in combined.items():
    item['final_score'] = 0.6 * item['dense'] + 0.4 * item['sparse']

final = sorted(combined.values(), key=lambda x: x['final_score'], reverse=True)

return final

# ===== USAGE =====
if __name__ == "__main__":
    # Initialize
    retrievers = init_hybrid_retriever()

    # Query
    query = "My product broke. Can I get a refund?"
    results = hybrid_retrieve(query, retrievers, domain="consumer")

```

```
# Display
for i, result in enumerate(results):
    print(f"\n--- Result {i+1} (score: {result['final_score']:.3f}) ---")
    print(f"Source: {result['metadata'].get('source', 'N/A')}")
    print(f"Section: {result['metadata'].get('section', 'N/A')}")
    print(f"Text: {result['text'][:200]}...")
```

Part 7: Debugging Hybrid Search

Issue 1: Wrong Domains Retrieved

```
python

# Add domain filtering
results = hybrid_retrieve(
    query="My salary hasn't been paid",
    retrievers=retrievers,
    domain="labour", # ← Filter to only Labour law
    top_k=4
)
```

Issue 2: Dense Search Retrieving Irrelevant Documents

```
python
```

```
# Check if embedding model is good
query = "product defect refund"
query_vec = embed_model.encode(query)

# Find most similar chunks manually
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Get all embeddings from ChromaDB
all_embeddings = [...] # retrieve from collection
sims = cosine_similarity([query_vec], all_embeddings)[0]
print(f"Max similarity: {sims.max():.3f}")
print(f"Mean similarity: {sims.mean():.3f}")

# If max < 0.7, embedding model may not be domain-specific
# Solution: Fine-tune embedding model on legal corpus
```

Issue 3: BM25 Retrieving Too Many Irrelevant Matches

```
python

# Increase BM25 threshold
scores = bm25.get_scores(query_tokens)

# Only keep scores above threshold
threshold = 5.0
top_indices = [i for i, s in enumerate(scores) if s > threshold][:5]

# Or adjust BM25 parameters (k1, b)
bm25_custom = BM25Okapi(tokenized_corpus, k1=2.0, b=0.5)
```

Summary Table: Dense vs Sparse vs Hybrid

Aspect	Dense	Sparse	Hybrid
Semantic Understanding	✓ ✓ ✓	✗	✓ ✓ ✓
Exact Keyword Match	✗	✓ ✓ ✓	✓ ✓ ✓
Section Number Retrieval	✗	✓ ✓ ✓	✓ ✓ ✓

Aspect	Dense	Sparse	Hybrid
Synonym Handling	✓ ✓ ✓	✗	✓ ✓ ✓
Query Latency	0.2-1s	<0.1s	0.3-1.5s
Memory (for 1000 chunks)	768MB embeddings	<1MB	769MB total
Maintenance	Embedding model	Tokenizer + BM25	Both

Recommendation: Start with Hybrid (current), optimize to Dense + HNSW only if latency becomes critical.

Production Checklist

- ☐ ChromaDB persisted to disk ((./chroma_db/))
- ☐ BM25 index saved to disk ((bm25_index.pkl))
- ☐ Domain filtering working ((where={"domain": "consumer"}))
- ☐ Weights tuned for use case (0.6 dense, 0.4 sparse by default)
- ☐ Test queries covering all 6 legal domains
- ☐ Latency measured (target: <1.5s total retrieval)
- ☐ Memory profiling (target: <2GB during inference)
- ☐ Fallback: if ChromaDB fails, use sparse-only BM25

Next Steps:

1. Integrate retriever output into LLM prompt (RAG context)
2. Fine-tune LLM on retrieved context (see (fine_tuning_flow.md))
3. Measure end-to-end latency and accuracy
4. Optimize weights based on domain-specific performance